Name : Prabhkirat Singh Marwah

Class : B

Batch : B4

Roll no. : 47

ml lab 5

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (train_test_split, GridSearchCV, cross_val_score, StratifiedKFold)
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import( accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report, roc_curve, roc_auc_score)

import warnings
warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
```

In [2]:
```
df = pd.read_csv("diabetes.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

In [4]:
```
df.tail()
```

Out[4]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

In [5]:
```
df.shape
```

Out[5]:
```
(768, 9)
```

In [6]:
```
df.info
```

Out[6]:
```
<bound method DataFrame.info of      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]>
```

In [7]:
```
df.describe()
```

Out[7]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

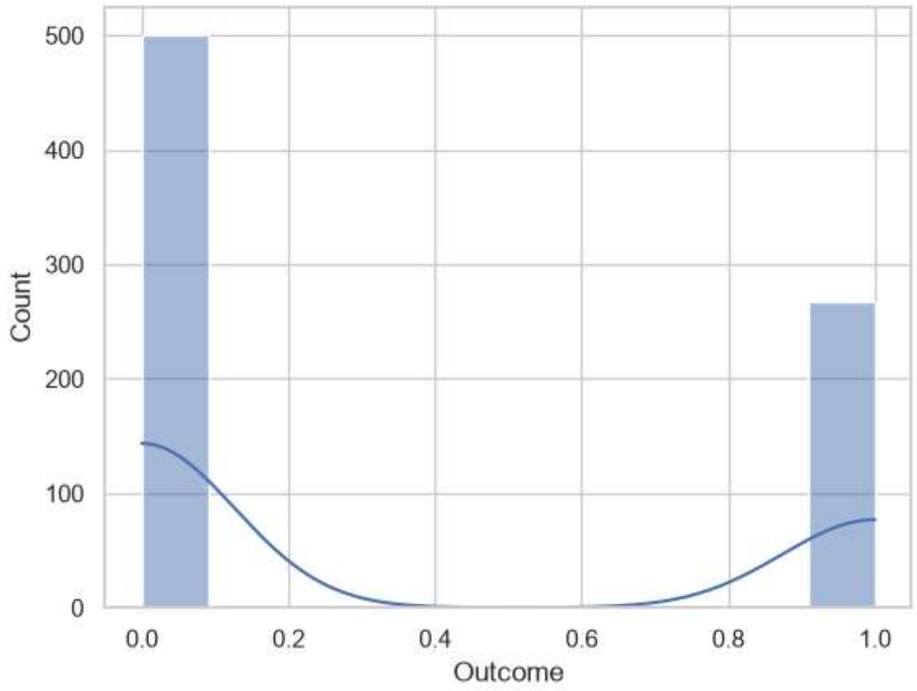
In [8]:
```
df.isnull().sum()
```

Out[8]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [9]:
```
df.duplicated().sum()
```

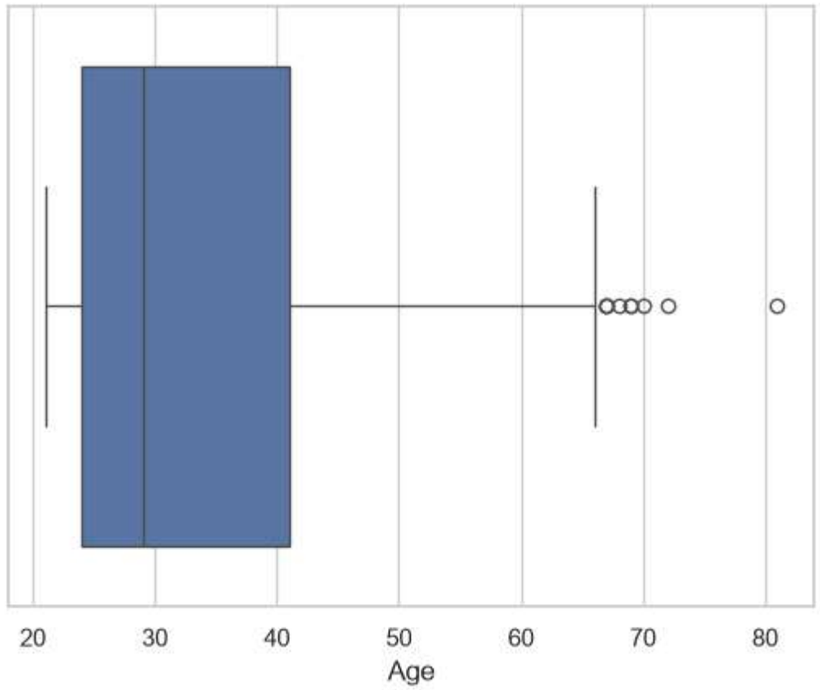
Out[9]:
```
np.int64(0)
```

In [10]:
```
sns.histplot(df["Outcome"], kde = True)
plt.show()
```



In [11]:
```
sns.boxplot(x=df["Age"])
plt.show
```

Out[11]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



In [12]:
```
plt.figure(figsize=(7,5))

plt.pie
```

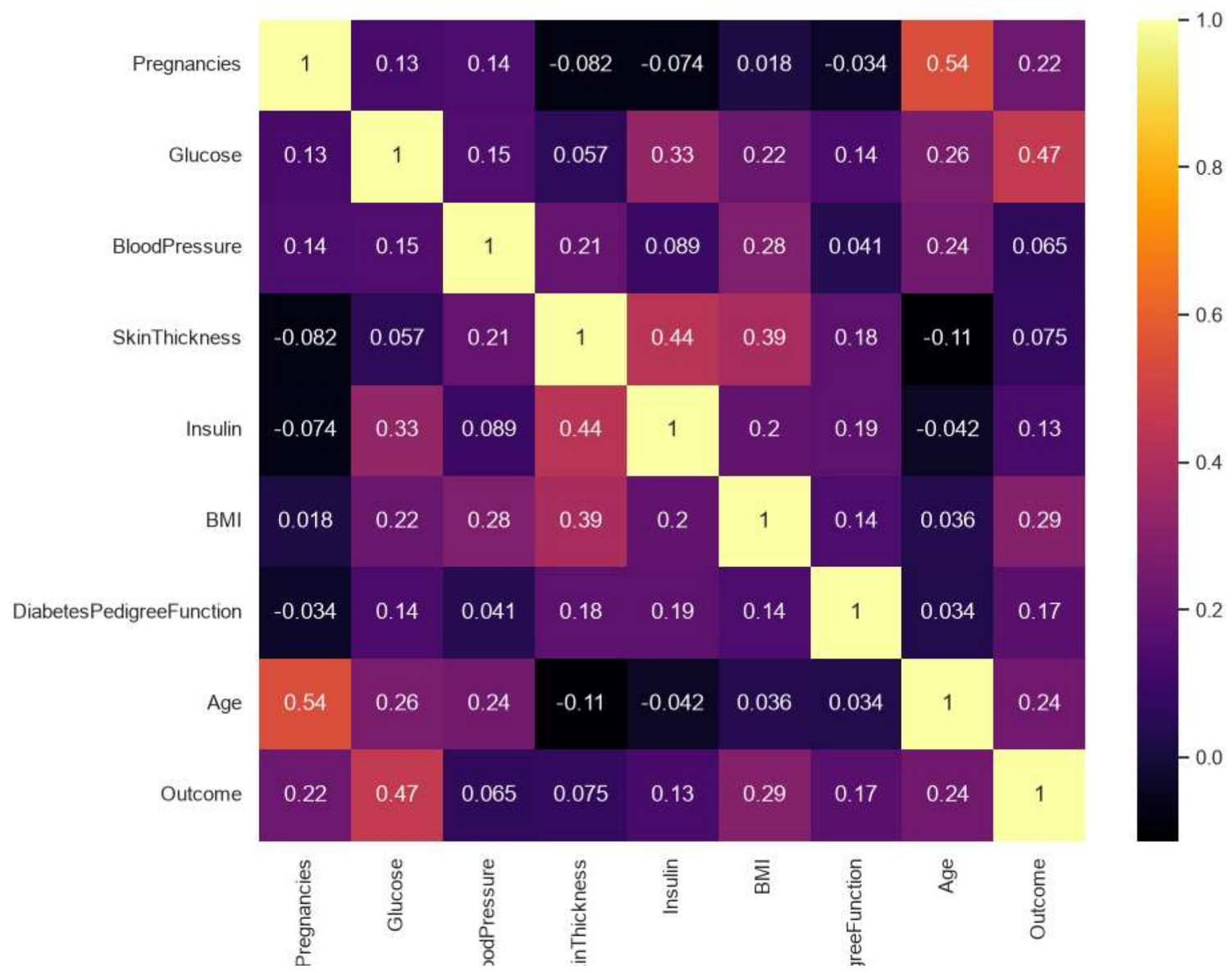
Out[12]:
```
<function matplotlib.pyplot.pie(x: 'ArrayLike', *, explode: 'ArrayLike | None' = None, labels: 'Sequence[str] | None' = None, colors: 'ColorType | Sequence[ColorType] | None' = None, autopct: 'str | Callable[[float], str] | None' = None, pctdistance: 'float' = 0.6, shadow: 'bool' = False, labeldistance: 'float | None' = 1.1, startangle: 'float' = 0, radius: 'float' = 1, counterclock: 'bool' = True, wedgeprops: 'dict[str, Any] | None' = None, textprops: 'dict[str, Any] | None' = None, center: 'tuple[float, float]' = (0, 0), frame: 'bool' = False, rotatelabels: 'bool' = False, normalize: 'bool' = True, hatch: 'str | Sequence[str] | None' = None, data: 'DataParamType' = None) -> 'PieContainer'>
<Figure size 700x500 with 0 Axes>
```

In [13]:
```
plt.figure(figsize=(10,8))

numeric_df = df.select_dtypes(include=['number'])
```

```
sns.heatmap(numeric_df.corr(),
            annot = True,
            cmap='inferno')

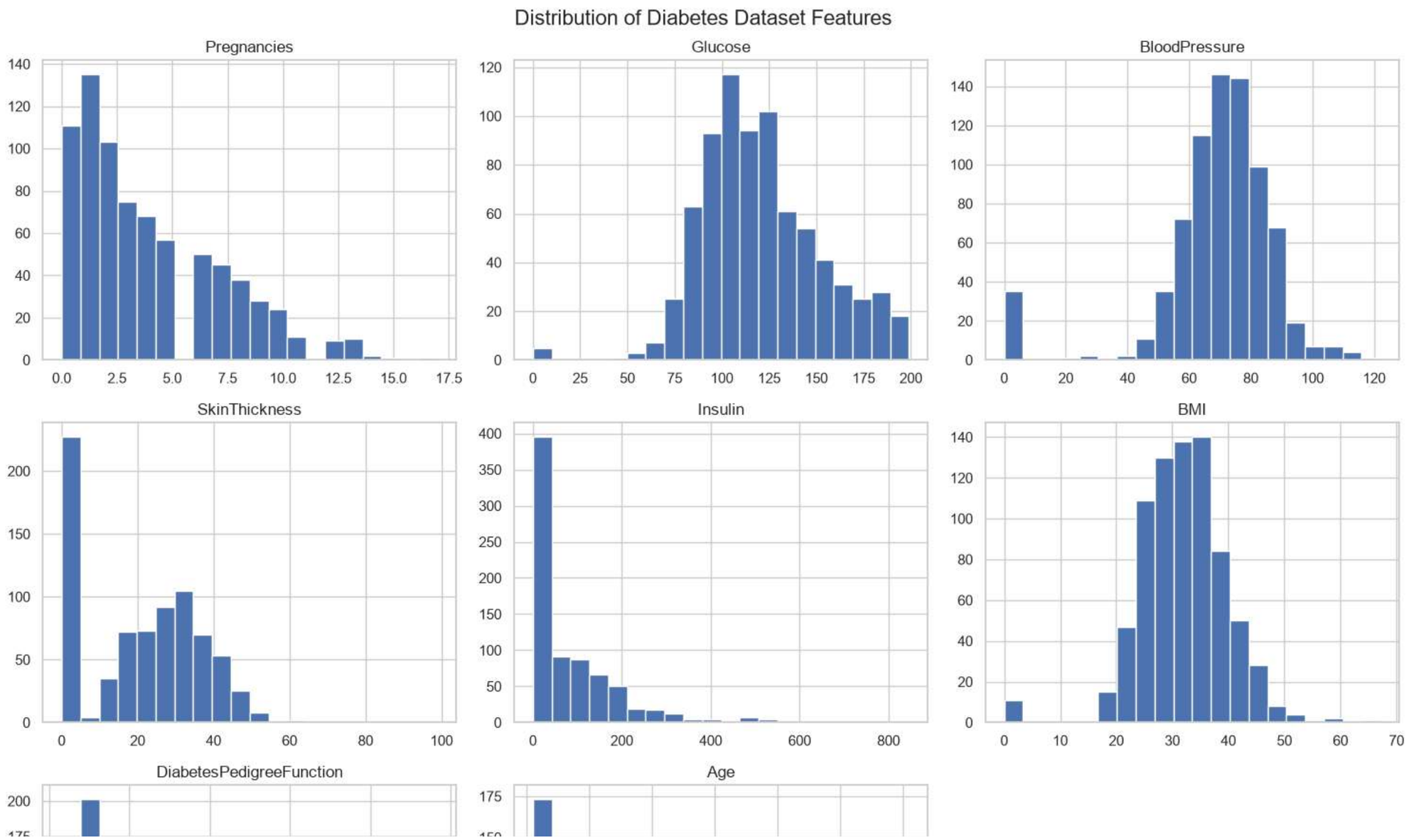
plt.show()
```

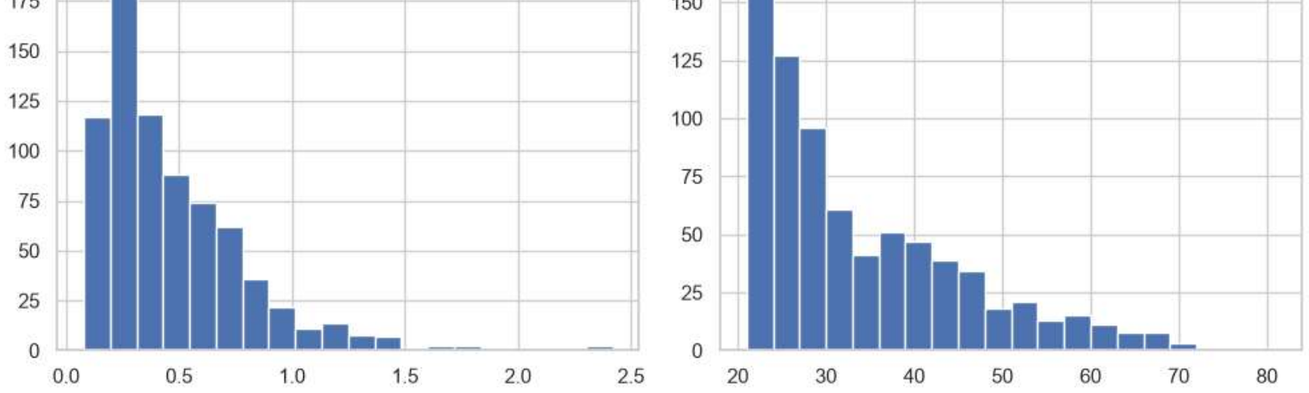

Pregnancies 1 0.13 0.14 -0.082 -0.074 0.018 -0.034 0.54 0.22
Glucose 0.13 1 0.15 0.057 0.33 0.22 0.14 0.26 0.47
BloodPressure 0.14 0.15 1 0.21 0.089 0.28 0.041 0.24 0.065
SkinThickness -0.082 0.057 0.21 1 0.44 0.39 0.18 -0.11 0.075
Insulin -0.074 0.33 0.089 0.44 1 0.2 0.19 -0.042 0.13
BMI 0.018 0.22 0.28 0.39 0.2 1 0.14 0.036 0.29
DiabetesPedigreeFunction -0.034 0.14 0.041 0.18 0.19 0.14 1 0.034 0.17
Age 0.54 0.26 0.24 -0.11 -0.042 0.036 0.034 1 0.24
Outcome 0.22 0.47 0.065 0.075 0.13 0.29 0.17 0.24 1
Pregnancies
Glucose
odPressure
inThickness
Insulin
BMI
greeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0



In [14]:
```
features = df.drop(columns="Outcome").columns
df[features].hist(figsize=(15,12),bins=20)
plt.suptitle("Distribution of Diabetes Dataset Features",fontsize=16)

plt.tight_layout()
plt.show()
```


Distribution of Diabetes Dataset Features
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age



In [15]:
```
zero_columns = ["Glucose","BloodPressure","SkinThickness", "Insulin", "BMI"]

zero_counts = (df[zero_columns] == 0).sum()

print("Zero values:")
print(zero_counts)
```

```
Zero values:
Glucose            5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI               11
dtype: int64
```

In [16]:
```
df_clean = df.copy()
df_clean[zero_columns] = df_clean[zero_columns].replace(0,np.nan)
print("Missing values after replacing invalid zeroes")
print(df_clean.isnull().sum())
```

```
Missing values after replacing invalid zeroes
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
Insulin                   374
BMI                        11
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [17]:
```
for col in zero_columns:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

print("Missing values after median imputation: ")
print(df_clean.isnull().sum())
```

```
Missing values after median imputation:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

In [18]:
```
X = df_clean.drop("Outcome", axis = 1)
y = df_clean["Outcome"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (768, 8)
y shape: (768,)
```

In [19]:
```
X_train,X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state = 42 , stratify=y)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 614
Testing samples: 154
```

In [20]:
```
print("Training Distribution:")
print(y_train.value_counts(normalize=True))

print("\nTesting distribution:")
print(y_test.value_counts(normalize=True))
```

```
Training Distribution:
Outcome
0    0.651466
1    0.348534
Name: proportion, dtype: float64

Testing distribution:
Outcome
0    0.649351
1    0.350649
Name: proportion, dtype: float64
```

In [21]:
```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [22]:
```
scaler.fit_transform(X_test)
```

Out[22]:
```
array([[ 0.85252995,  1.16484536, -0.8288433 , ..., -0.71933141,
        -0.46562514,  0.63400979],
       [ 1.69054875, -1.66920722,  2.88630463, ...,  0.42561436,
        -0.49253293,  1.24533587],
       [-0.54416805,  0.01253827,  0.23262754, ...,  0.48215489,
         0.09943846, -0.58864236],
       ...,
       [-0.54416805, -1.23319913, -1.89031414, ..., -0.56384494,
         3.73497988, -0.67597466],
       [ 0.01451115,  1.94343123,  0.40953934, ...,  0.63764135,
        -0.55531777, -0.15198088],
       [-0.82350765, -1.57577692,  0.40953934, ...,  0.1005063 ,
        -0.08293657, -1.02530385]], shape=(154, 8))
```

In [23]:
```
scaler.transform(X_test)
```

Out[23]:
```
array([[ 0.85252995,  1.16484536, -0.8288433 , ..., -0.71933141,
        -0.46562514,  0.63400979],
       [ 1.69054875, -1.66920722,  2.88630463, ...,  0.42561436,
        -0.49253293,  1.24533587],
       [-0.54416805,  0.01253827,  0.23262754, ...,  0.48215489,
         0.09943846, -0.58864236],
       ...,
       [-0.54416805, -1.23319913, -1.89031414, ..., -0.56384494,
         3.73497988, -0.67597466],
       [ 0.01451115,  1.94343123,  0.40953934, ...,  0.63764135,
        -0.55531777, -0.15198088],
       [-0.82350765, -1.57577692,  0.40953934, ...,  0.1005063 ,
        -0.08293657, -1.02530385]], shape=(154, 8))
```

In [24]:
```
comparison = pd.DataFrame({
    "Original MEan" : X_train.mean(),
    "Original std": X_train.std(),
    "Scaled Mean": X_train_scaled.mean(axis = 0),
    "Scaled std": X_train_scaled.std(axis = 0)
})
```

In [25]:
```
k = 5
```

In [26]:
```
knn = KNeighborsClassifier(n_neighbors = 5)
knn.fit(X_train_scaled, y_train)
```

Out[26]:

▾ KNeighborsClassifier

▸ Parameters

▸ Fitted attributes

In [27]:
```
y_pred = knn.predict(X_test_scaled)

print("Predictions:")
print(y_pred[:20])
```

```
Predictions:
[1 0 0 1 0 0 0 1 0 1 0 1 0 0 0 1 1 0 1 0]
```

In [28]:
```
y_prob = knn.predict_proba(X_test_scaled)[:, 1]

print(y_prob[:20])
```

```
[0.6 0.2 0.2 0.6 0.  0.2 0.2 0.8 0.  0.6 0.4 0.6 0.  0.  0.  0.6 0.8 0.
 1.  0.2]
```

In [29]:
```
accuracy = accuracy_score(y_test,y_pred)
print(f"Accuracy: {accuracy:.4f}")
```

```
Accuracy: 0.7532
```

In [30]:
```
recall = recall_score(y_test,y_pred)
print(f"Recall: {recall:.4f}")
```

```
Recall: 0.6111
```

In [31]:
```
f1 = f1_score(y_test, y_pred)
print(f"F1 score: {f1:.4f}")
```

```
F1 score: 0.6346
```

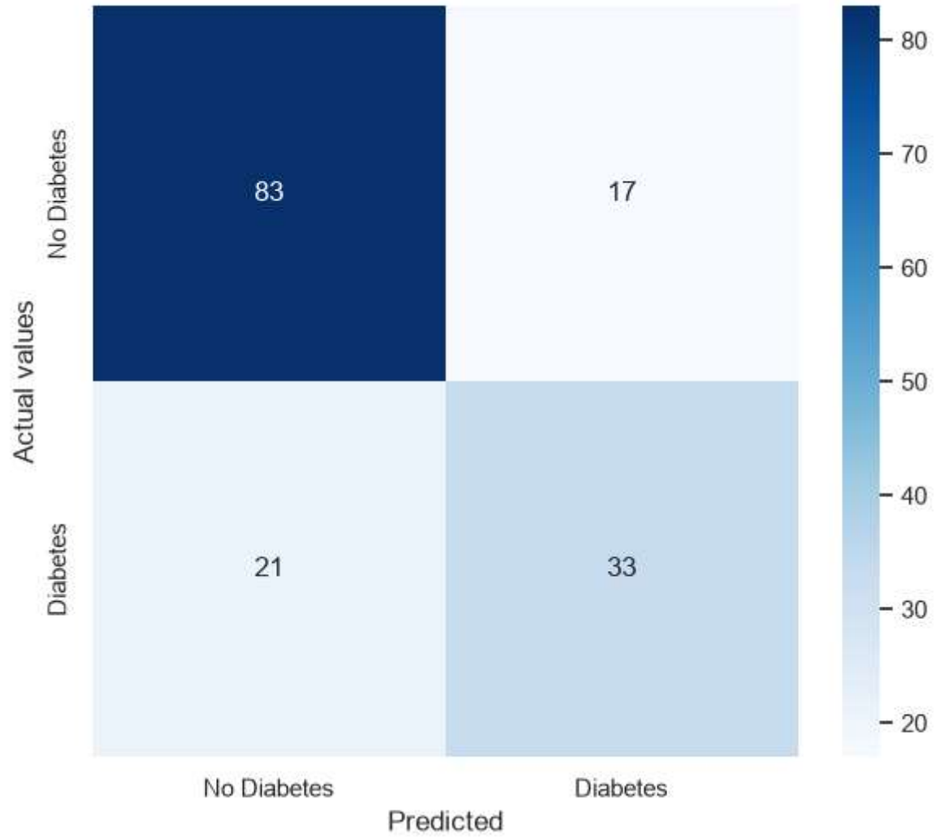
In [32]:
```
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[83 17]
 [21 33]]
```

In [33]:
```
plt.figure(figsize=(7,6))

sns.heatmap(cm, annot = True, fmt="d", cmap="Blues", xticklabels = ["No Diabetes", "Diabetes"],
            yticklabels = ["No Diabetes", "Diabetes"]
)

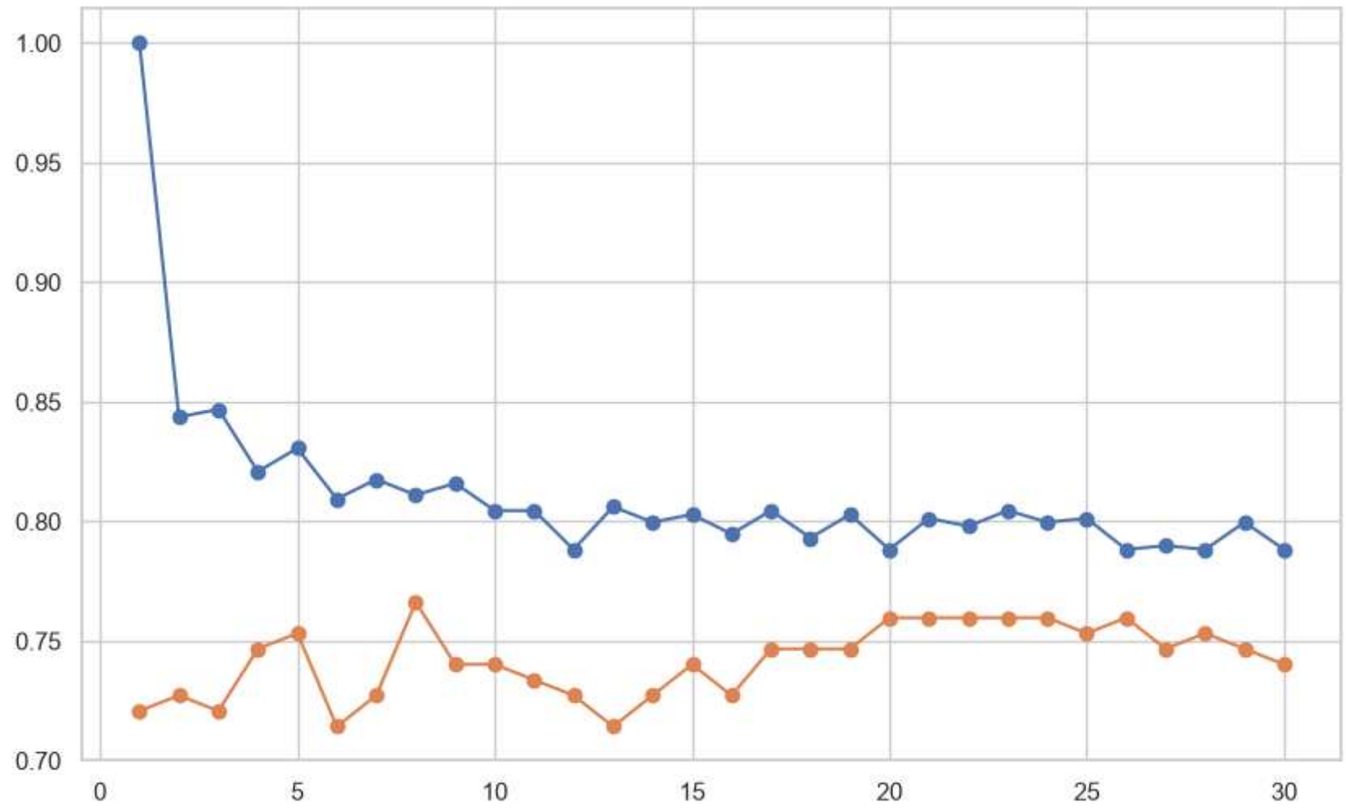
plt.xlabel("Predicted")
plt.ylabel("Actual values")
plt.show()
```



In [34]:
```
k_values = range(1, 31)
train_accuracy, test_accuracy = [], []
for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled, y_train)
    train_accuracy.append(model.score(X_train_scaled, y_train))
    test_accuracy.append(model.score(X_test_scaled, y_test))
```

In [35]:
```
plt.figure(figsize=(10,6))
plt.plot(k_values, train_accuracy, marker='o', label='Training accuracy')
plt.plot(k_values, test_accuracy, marker='o', label='Testing accuracy')
```

Out[35]: [<matplotlib.lines.Line2D at 0x1f408e7f390>]

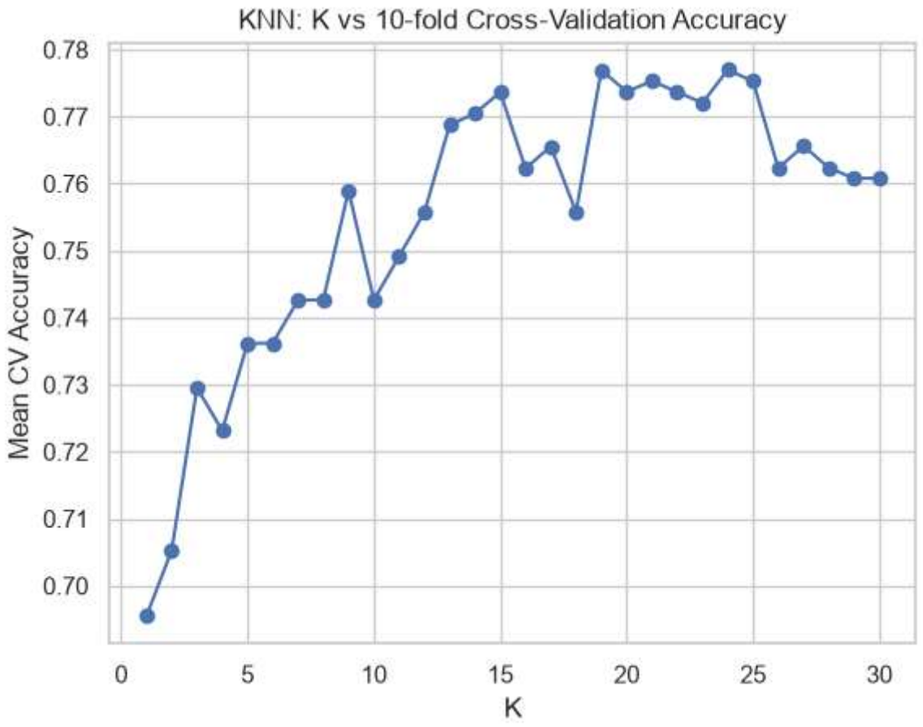


In [37]:
```
best_k_accuracy = k_values[np.argmax(test_accuracy)]
print('Best K:', best_k_accuracy)
print('Best Accuracy:', max(test_accuracy))
```

```
Best K: 8
Best Accuracy: 0.7662337662337663
```

In [38]:
```
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
cv_scores = []
for k in range(1, 31):
    model = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='accuracy')
    cv_scores.append(scores.mean())
```

In [39]:
```
plt.plot(range(1, 31), cv_scores, marker='o')
plt.xlabel('K'); plt.ylabel('Mean CV Accuracy')
plt.title('KNN: K vs 10-fold Cross-Validation Accuracy')
plt.show()
```

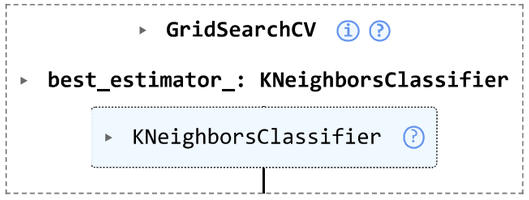


In [40]:
```
best_k_cv = np.argmax(cv_scores) + 1
print('Best K based on cross validation:', best_k_cv)
print('Best cv Accuracy:', max(cv_scores))
```

```
Best K based on cross validation: 24
Best cv Accuracy: 0.776969857218403
```

In [41]:
```
param_grid = {
    'n_neighbors': list(range(3, 31, 2)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)
```

Out[41]:



In [42]:
```
print('Best Parameters:', grid_search.best_params_)
print('Best Cross-Validation Accuracy:', grid_search.best_score_)
```

```
Best Parameters: {'metric': 'euclidean', 'n_neighbors': 19, 'weights': 'uniform'}
Best Cross-Validation Accuracy: 0.7769025723044115
```

In [ ]: